

Group 13

E/21/050 - Bandara H.B.C.T.

E/21/052 - Bandara H.M.P.D.

CO503 - Advanced Embedded Systems

Practical 3: Multi-Processor System-on-Chip (MPSoC) Design

1. Introduction

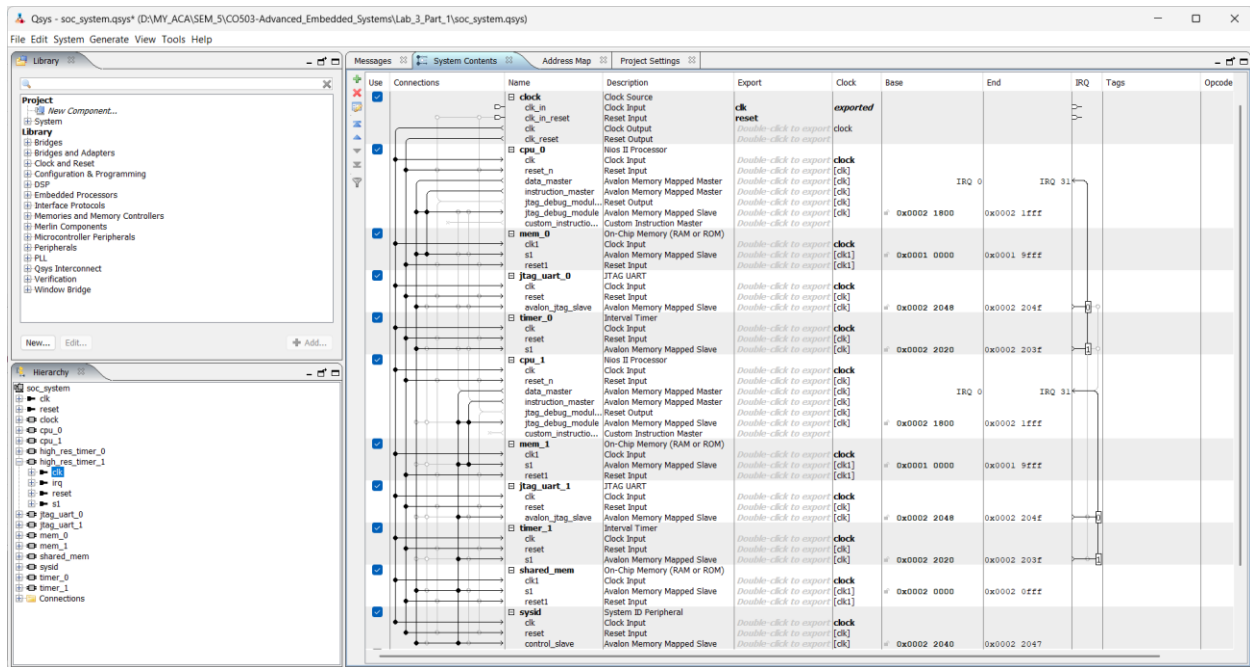
This practical was aimed to design and test a Multi-Processor System-on-Chip (MPSoC) implemented on an FPGA. The task involved setting up inter-processor communication between two cores of the Nios II by applying producer-consumer application model. We used and tested two dissimilar architectures of communication, a software FIFO queue that manages within shared memory partition and a hardware FIFO memory core. We used these two approaches to evaluate the effects of hardware acceleration on the multiprocessor communication throughput and processing time.

2. Experimental Setup and Design Procedure

Part 1: Shared Memory Software FIFO

We started by coming up with the first MPSoC hardware designed in Qsys. We instantiated two Nios II/e processors (cpu_0 and cpu_1). To maintain independent execution, each processor was provided with its own private instruction memory, JTAG UART and interval timer. To communicate, we added a shared On-Chip Memory block which was connected to the data masters of both processors. Once we had assigned base addresses and resolved an IRQ conflict we then generated the HDL and successfully compiled the hardware in Quartus.

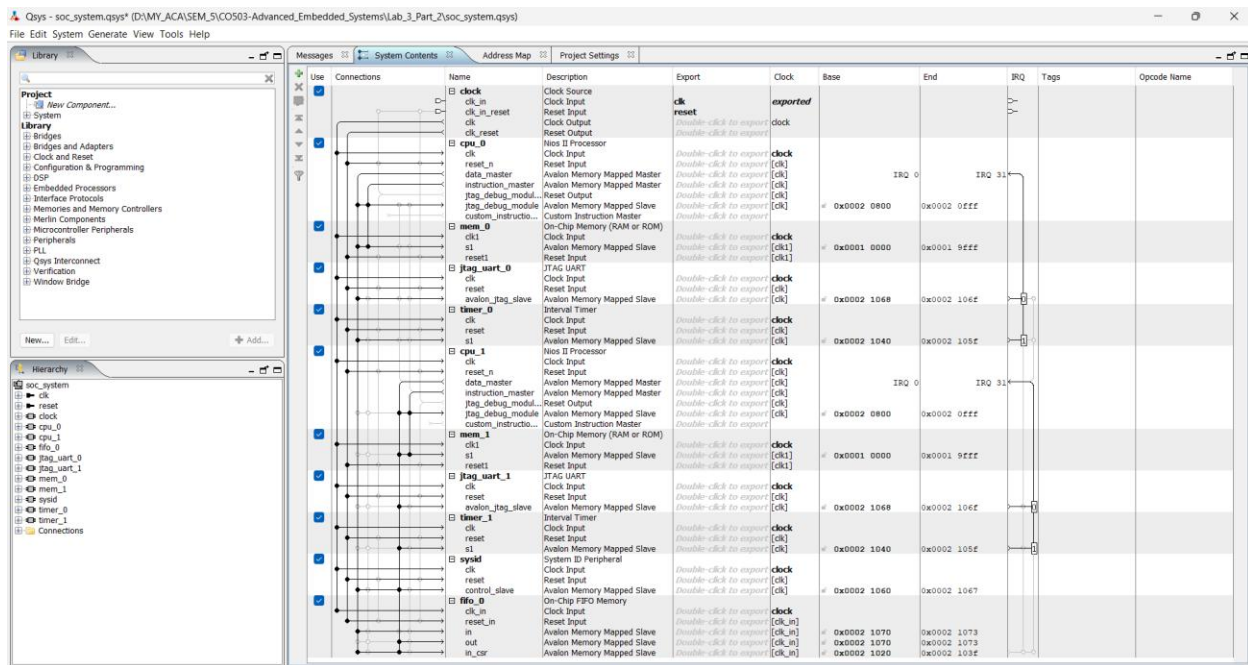
We were able to generate two Board Support Packages (BSPs) in the Eclipse environment. We used a software FIFO queue (ring buffer) within the shared partition. Data tokens were written by the producer (cpu_0) and "Full" and "Empty" pointer was updated manually and the consumer (cpu_1) polled these flags via the system interconnect to read the data.



SoC hardware design of Software FIFO in Qsys

Part 2: Dedicated Hardware FIFO

The second phase was to return to Qsys and remove the shared memory block replacing it with an Altera On-Chip FIFO Memory component so the CPUs no longer had to handle queue management. We configured the FIFO to single-clock mode and an Avalon-MM write slave connected exclusively to the CPU_0 and an Avalon-MM read slave connected exclusively to the CPU_1. Importantly, we configured single input status interface (CSR) and connected it to the two processors. This enabled the Producer and Consumer to read directly the hardware FPGA fabric FULL (Bit 0), EMPTY (Bit 1) status bits without software pointers. We then recompiled the hardware and updated the BSPs to use the new memory map.



SoC hardware design of Hardware FIFO in Qsys

3. Actual Problems Faced and Troubleshooting

3.1 Hardware FIFO Port Visibility and Compilation Errors

When adapting the C codebase for the hardware FIFO in Part 2 we faced compilation errors which said that the macros like FIFO_0_IN_BASE and ALTERA_AVALON_FIFO_STATUS_FULL_MSK had not been declared in the consumer project. This was due to the fact that our Qsys connections were strictly unidirectional, the Consumer had only an out port but no visibility of the input hardware macros.

- Solution:** This was resolved in a clean way by adding preprocessor directives of the form `#ifdef` in our FIFO_1.c file. This shielded the compiler to be able to dynamically add the write function just to cpu_0 (where FIFO_0_IN_BASE had been defined) and the read function just to cpu_1.

3.2 Eclipse Run Configurations and System Resets

During testing we noticed that the system would sometimes hang or the shared memory location be wiped when the Producer had already initialized the shared memory location.

- **Solution:** We found out that when the Consumer processor triggered a system-wide reset when starting up, it wiped out the hardware flags set by the Producer. We overcame this by carefully managing the Run Configurations in Eclipse. Inside the Run Configurations we explicitly unchecked “Reset the processor” when we were starting up the applications, so that the shared memory flags remained intact across both cores.

3.3 Memory Footprint, C99 Mode, and Library Limitations

In order to monitor the execution times, we have inserted a loop of 100,000 iterations. However, applications that used standard printf functions had sizes that overflow the 40 KB private instruction memories. This was solved by enabling the option of the “Small C library” in the BSP. This, however, introduced a sequence of secondary problems. First, we got errors in compilation stating that “for loop initial declarations are only allowed in C99 mode”. We solved this problem by declaring our loop variables outside the for loop parameters. Then we have discovered that the Small C Library does not support the fflush() function and the printing of 64-bit integer (%llu).

- **Solution:** To avoid this, we used the normal 32-bit arithmetic to calculate the system timer and this was safe because the clock speed was 50 MHz.

3.4 JTAG UART Disconnects and Benchmarking Constraints

In our high-speed benchmarking, the Producer processor consistently failed to display its final execution time to the Eclipse console. We eliminated the processor from the list of possible causes, because we suspected that the processor was going into its infinity while(1) halt loop before JTAG hardware could completely flush the print buffer over the USB cable.

- **Solution:** Since fflush() was not there, we cleverly bypassed the broken console. We also programmed the Producer to compute its final execution time and push that into the FIFO as the 100,001st token and this would allow the Consumer to safely print the results of the two processors in a single console window.

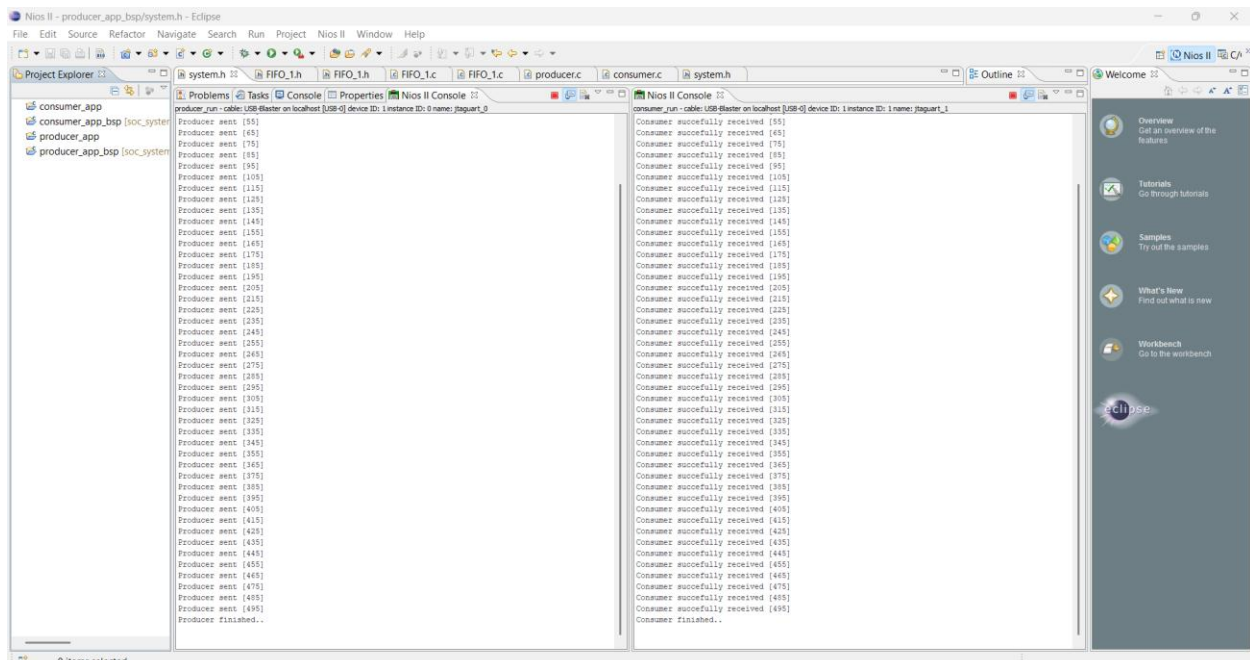
3.5 Synchronization and “Ghost Data”

When trying to measure actual throughput we had realized that human reaction time (the delay in manually clicking 'Run' for the second processor) was heavily distorting the stopwatch of the Producer. We tried to run the Consumer first to wait for the Producer but the Consumer immediately read 100,000 dirty tokens that remained in the physical RAM from previous runs, which is a 'ghost data' synchronization problem.

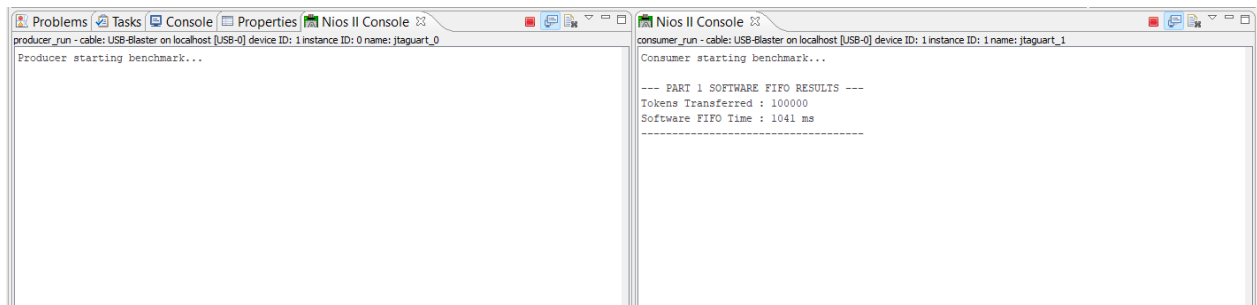
- **Solution:** We configured the Producer to initialize and clear the memory (FIFO_1_INIT(1)) and the Consumer to simply wait without clearing (FIFO_1_INIT(0)) in order to have a perfect benchmark without human reaction time and ghost data. We then ran the Producer first. The Producer immediately filled the buffer and physically froze waiting for the Consumer. When we ran the Consumer, we were able to trigger both stopwatches simultaneously, which took the exact time of data drain when active and recorded this perfectly.

4. Results and Observations

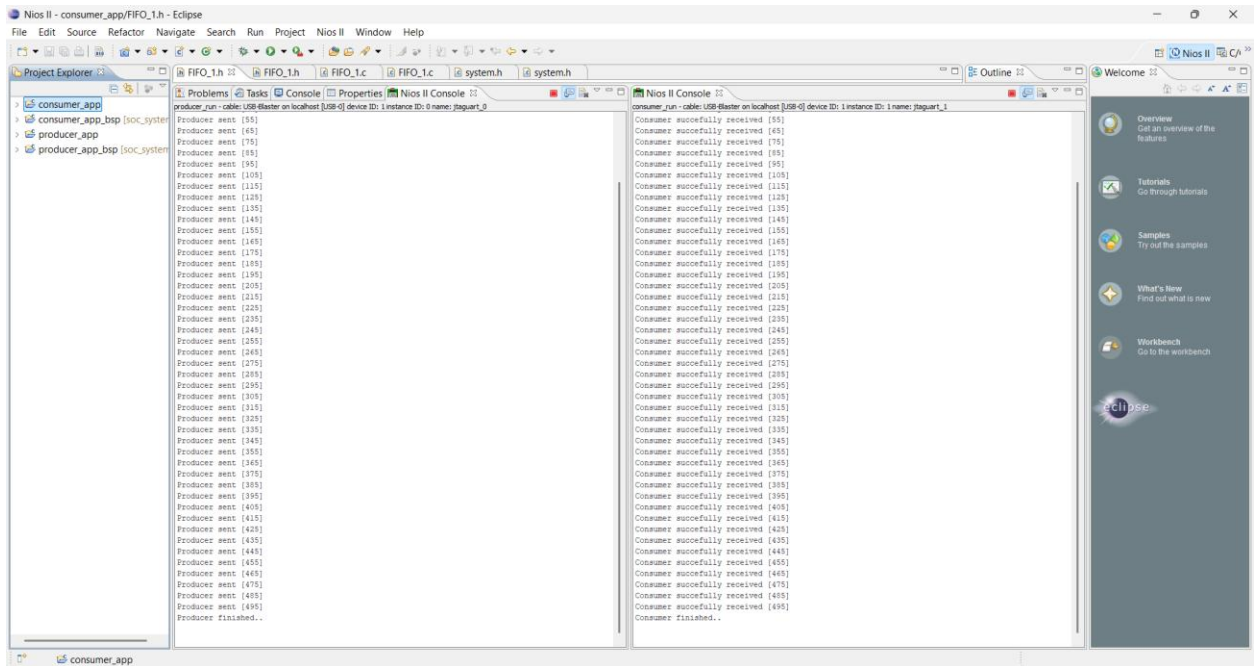
To strictly evaluate the communication performance without I/O overhead, we modified the application to transmit a bulk payload of 100,000 integer tokens continuously, removing all `printf` and `delay` statements from the tight loop. We utilized the system high-resolution timestamp timers to measure the execution time.



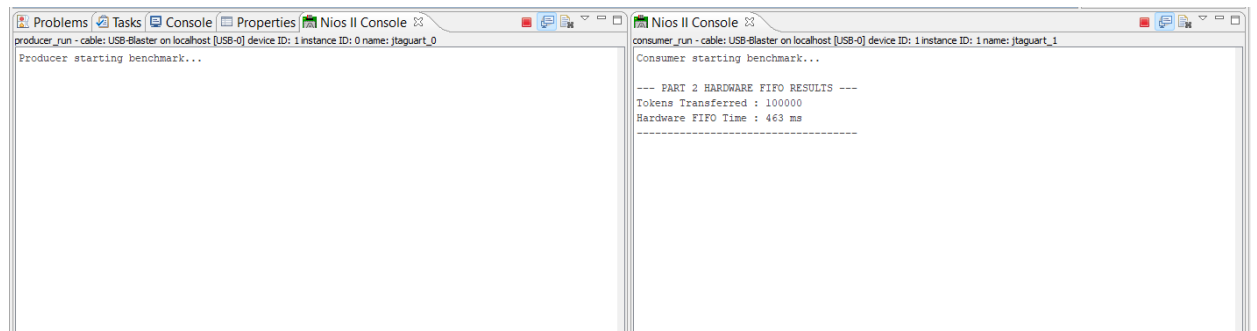
Console output verifying Software FIFO data transmission



Benchmark test results for Software FIFO



Console output verifying Hardware FIFO data transmission



Benchmark test results for Hardware FIFO

Table 1: Performance Comparison (100,000 Tokens)

Implementation	Execution Time (ms)	Throughput (Tokens/sec)
Software FIFO (Shared Mem)	1041	96,061
Hardware FIFO Core	463	215,982

Speedup Ratio:

- Hardware FIFO vs Software FIFO: **2.25x**

As expected, the hardware FIFO showed a massive performance advantage, achieving more than double the throughput. In the Software FIFO architecture, the processors wasted multiple clock cycles on overhead computing pointer arithmetic, tracking buffer capacities and continuously reading/writing synchronization flags across the system interconnect. The Hardware FIFO architecture used internal ring buffer and flags which were automatically handled by a dedicated FPGA silicon. The processors simply pushed or pulled data to a single static memory address, drastically increasing the communication bandwidth.

5. What We Learned

This practical gave an in-depth understanding of MPSoC design and inter-processor communication mechanisms. We have learnt how to properly partition hardware resources in Qsys to create private and shared domains for multiple processors. We gained hands-on experience with the complexities of software synchronization, particularly “ghost data” persistence in shared memory and JTAG UART timing constraints. Finally, implementing the hardware FIFO demonstrated the practical value of hardware/software co-design, proving that offloading control logic to dedicated FPGA hardware not only simplifies the software side but also significantly accelerates system performance.